OS -- Process and Threads Fundamentals

[Target] Target: Master process/thread concepts for MSTC Systems interview
Read time: 20 minutes

HIGH PRIORITY -- INTERVIEW CRITICAL
1. OPERATING SYSTEM DEFINITION YOU MISSED THIS IN MOCK
Technical Definition:
"An Operating System acts as an interface between computer hardware and user. It manages and controls all the hardware and flow of data, instructions and information within the system. It takes instructions from the user and directs it to CPU, which further passes the data to the hardware."
40s Script:
"An Operating System is system software that acts as an interface between users and computer hardware. It manages CPU, memory, files, and I/O devices. Key functions include process management, memory management, file system, and providing services like system calls. Examples include Windows, Linux, macOS. Without OS, users would need to directly interact with hardware using machine language."
Key Functions:
- Process Management (scheduling, creation, termination)
- Memory Management (allocation, paging, virtual memory)
- File Management (create, delete, read, write)
- I/O Management (device drivers, buffering)
- Security (access control, authentication)
- Networking (protocols, communication)
Types of Operating Systems:
1. Batch Operating System processes jobs in batches
2. Multi-programming Operating System multiple programs in memory
3. Multi-processing Operating System multiple CPUs
4. Multi-tasking Operating System time-sharing between tasks
5. Real-time Operating System guaranteed response times
6. Distributed Operating System manages multiple machines

2. PROCESS CONCEPT
Definition:
- Process = program in execution
- Program = passive entity (code on disk)
- Process = active entity (program loaded in memory with execution state)
Process Components:
1. Text Section program code (instructions)
2. Data Section global variables
3. Heap dynamically allocated memory during runtime
4. Stack temporary data (function parameters, return addresses, local variables)
Memory Layout:
High Address
| Stack | grows downward
| |
| Heap | grows upward
| Data | global variables
| Text | program code
Low Address

3. PROCESS STATES INTERVIEW FAVORITE
5 Process States:

New process being created
Running instructions being executed
Waiting waiting for some event (I/O completion)
Ready waiting to be assigned to processor
Terminated process finished execution

State Transitions:
New Ready (admitted)
Ready Running (scheduler dispatch)
Running Waiting (I/O or event wait)
Waiting Ready (I/O or event completion)
Running Ready (interrupt)
Running Terminated (exit)
40s Script:
"A process has 5 states. New means it's being created. Ready means it's loaded in memory waiting for CPU. Running means CPU is executing its instructions. Waiting means it's blocked on I/O or some event. Terminated means it's finished. The OS scheduler manages transitions between Ready and Running states."

4. PROCESS CONTROL BLOCK (PCB)
Definition:
Data structure maintained by OS for each process to store process information.

PCB Contains:
- Process ID (PID) unique identifier
- Process State new, ready, running, waiting, terminated
- Program Counter address of next instruction
- CPU Registers accumulator, index registers, stack pointers
- Memory Limits base and limit registers
- List of Open Files file descriptors
- Accounting Information CPU time used, time limits

Why PCB is Important:
- Context Switching save/restore process state
- Process Scheduling scheduler uses PCB information
- Memory Management tracks memory allocation per process

5. PROCESS vs THREAD YOU MISSED THIS DISTINCTION IN MOCK

PROCESS
Definition: Independent execution unit with its own memory space
Characteristics:
- Heavy-weight requires more resources
- Isolated each process has separate memory
- Communication through IPC (Inter-Process Communication)
- Creation Time slower (more overhead)
- Context Switch expensive (save/restore entire state)

THREAD
Definition: Light-weight process that shares memory space with other threads
Characteristics:
- Light-weight requires fewer resources
- Shared Memory threads in same process share memory
- Communication direct memory access (faster)
- Creation Time faster
- Context Switch cheaper (less state to save)

Comparison Table:

Aspect	Process	Thread
Memory	Separate address space	Shared address space
Communication	IPC (pipes, shared memory)	Direct memory access
Creation Time	Slow	Fast
Context Switch	Expensive	Cheap
Independence	Independent	Not independent
Crash Impact	Isolated (one crash != all crash)	One thread crash = process crash

40s Script:
"Process is an independent program execution with its own memory space. Thread is a lightweight unit within a process that shares memory with other threads. Key difference: processes are isolated with separate memory, threads share

memory within a process. This makes thread communication faster but less secure than process communication."

6. TYPES OF THREADS
A. USER-LEVEL THREADS (ULT)

Managed by User-level thread library
Kernel Awareness Kernel doesn't know about threads
Advantages Fast creation, no system call overhead
Disadvantages Blocking system call blocks entire process

B. KERNEL-LEVEL THREADS (KLT)

Managed by Operating System kernel
Kernel Awareness Kernel schedules threads directly
Advantages True parallelism on multi-core
Disadvantages Slower creation, system call overhead

C. HYBRID THREADING MODEL

Many-to-One Multiple user threads one kernel thread
One-to-One Each user thread separate kernel thread
Many-to-Many Multiple user threads multiple kernel threads

7. MULTITHREADING MODELS
Many-to-One Model:
- Multiple user threads mapped to single kernel thread
- Problem: If one thread blocks, entire process blocks
- Example: Green threads in older JVMs
One-to-One Model:
- Each user thread mapped to separate kernel thread
- Advantage: True parallelism, one thread block != process block
- Disadvantage: Overhead of creating kernel threads
- Example: Windows, Linux
Many-to-Many Model:
- Multiple user threads mapped to multiple kernel threads
- Advantage: Best of both worlds
- Disadvantage: Complex implementation

Quick Cheat Sheet
```
OS Definition:
  Interface between user and hardware
  Manages CPU, memory, files, I/O, security
Process States (5):
  New  Ready  Running  Waiting/Terminated
  Ready  Running (scheduler)
  Running  Waiting (I/O)
  Waiting  Ready (I/O complete)
Process vs Thread:
  Process  separate memory, independent, heavy
  Thread  shared memory, dependent, light
Thread Types:
  User-Level  managed by library, fast, blocking issue
  Kernel-Level  managed by OS, slow, true parallelism
PCB Contains:
  PID, State, Program Counter, Registers, Memory Limits, Open Files
```

Follow-up Questions & Answers
Q: What happens during context switching?
A: OS saves current process state (registers, program counter) in its PCB, then loads the saved state of next process from its PCB. This allows multiple processes to share single CPU.
Q: Why are threads called "lightweight processes"?
A: Because they require less overhead to create and manage compared to processes. Threads share memory, file descriptors, and other process resources, so less information needs to be maintained separately.
Q: Can threads improve performance on single-core systems?
A: Yes, through concurrency (not parallelism). While one thread waits for I/O, another thread can use CPU. This improves overall system utilization even on single-core systems.
Q: What is the difference between concurrency and parallelism?
A: Concurrency is dealing with multiple things at once (time-sharing on single CPU). Parallelism is doing multiple things simultaneously (multiple CPUs/cores).